

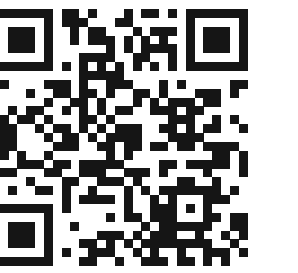
Succinct and Fast Tiny Pointer Hash Tables

Xilin Tang¹ Yuqi Mai¹ William Kuszmaul² Alex Conway³

¹Cornell University

²Carnegie Mellon University

³Cornell Tech

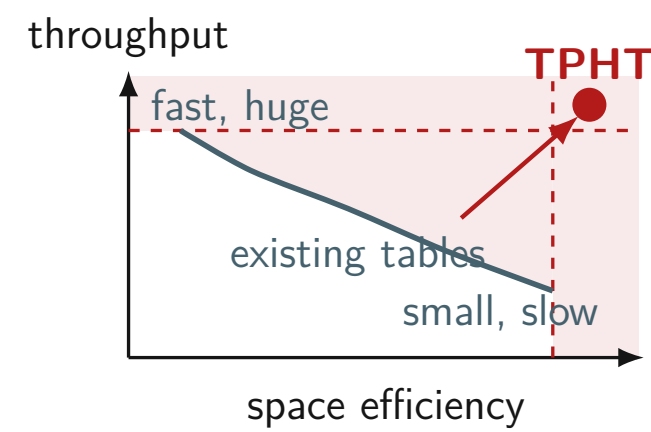


github.com/Xilinion/TPHT

TPHT breaks the speed–space trade-off: **smaller and faster than the state of the art** — even **smaller than the data itself**.

What is a good hash table?

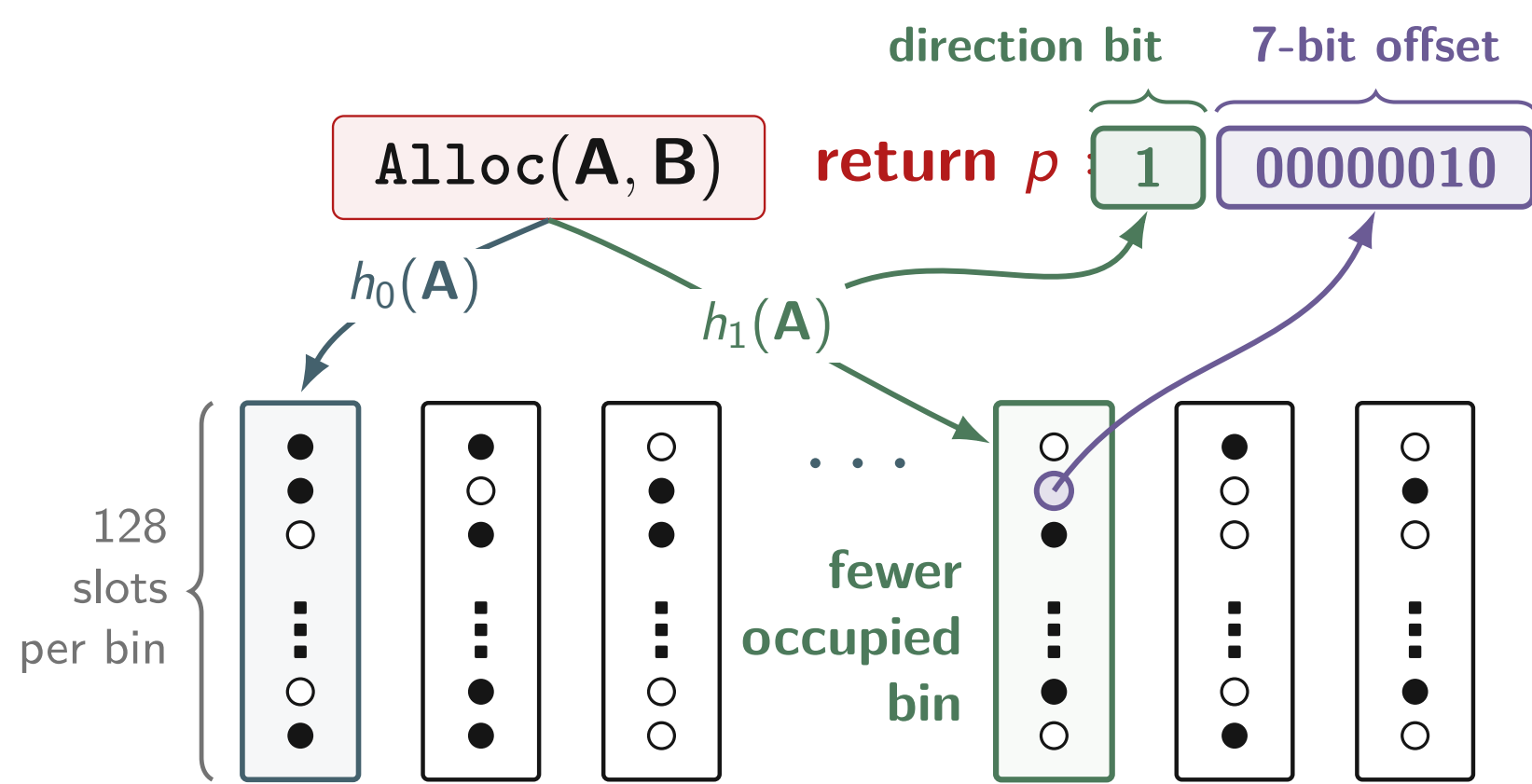
Every practical hash table trades throughput against memory: chaining pays for pointers, open addressing pays for empty slots. TPHT engineers two ideas from theory to escape the trade-off.



Two ingredients from theory

Ingredient 1: tiny pointers

A **tiny pointer** is decoded only with its unique **owner** — the owner supplies the context, so it shrinks to **one byte**.

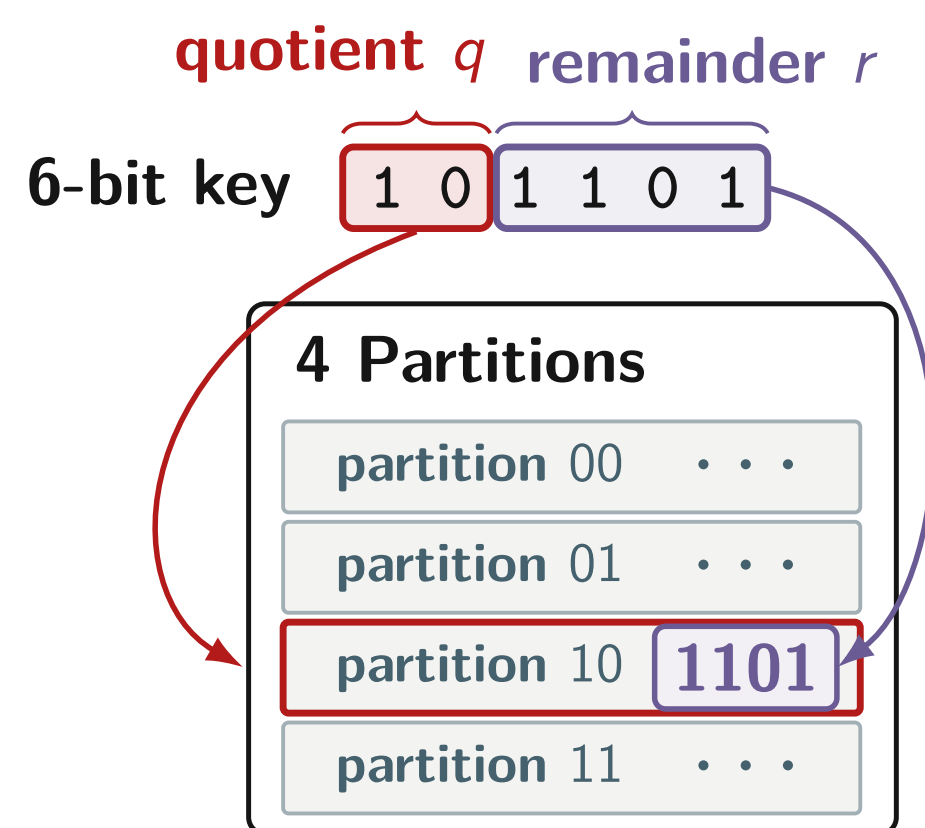


The *dereference table* hashes the owner to two candidate bins and picks the emptier — safe up to **95%** load.

Ingredient 2: quotienting

The quotient **selects** the partition; the location itself remembers q , so only r is written down.

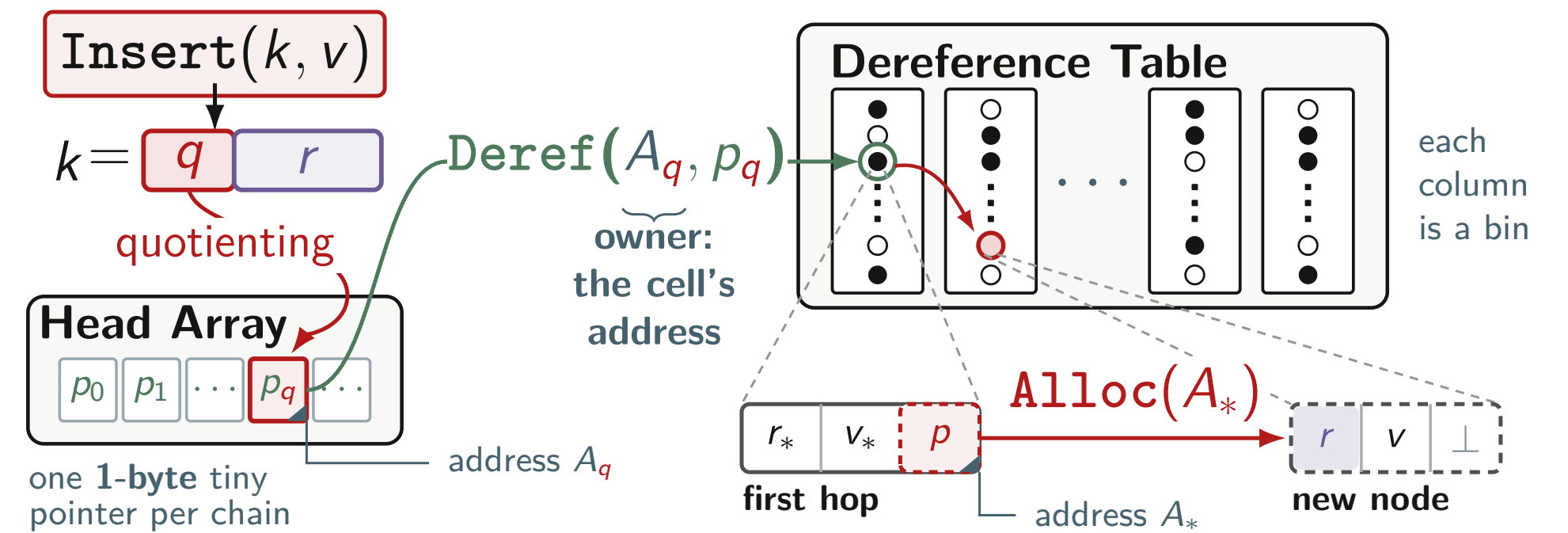
$k = q \circ r$ is recovered exactly via a one-round Feistel permutation.



The TPHT designs

Chained-TPHT

To our knowledge the **first practical succinct hash table**: footprint below the raw data size, constant-time operations.

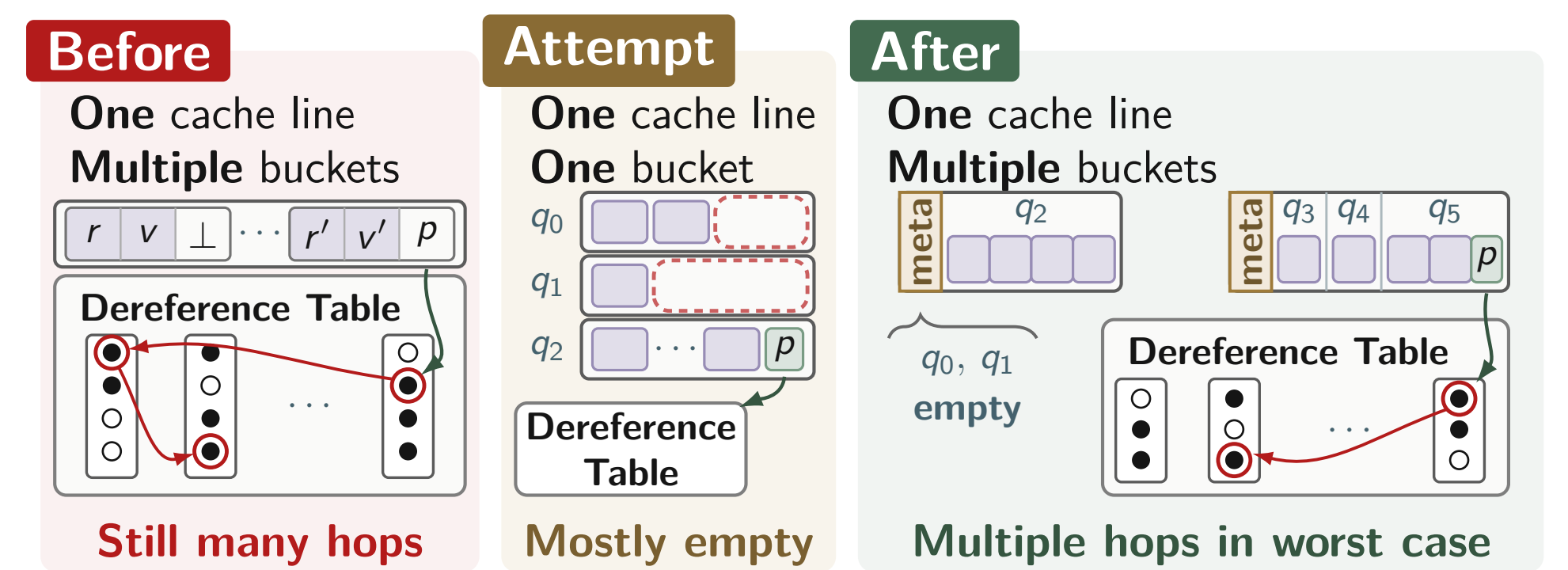


- Head array is **8×** smaller — heads stay in cache.
- The quotient is **never stored**; chains keep only (r, v, p) .
- First tuple inlined into the head entry: one fewer cache miss.

Making it faster: packing chains into cache lines

Lifting payload to head array;

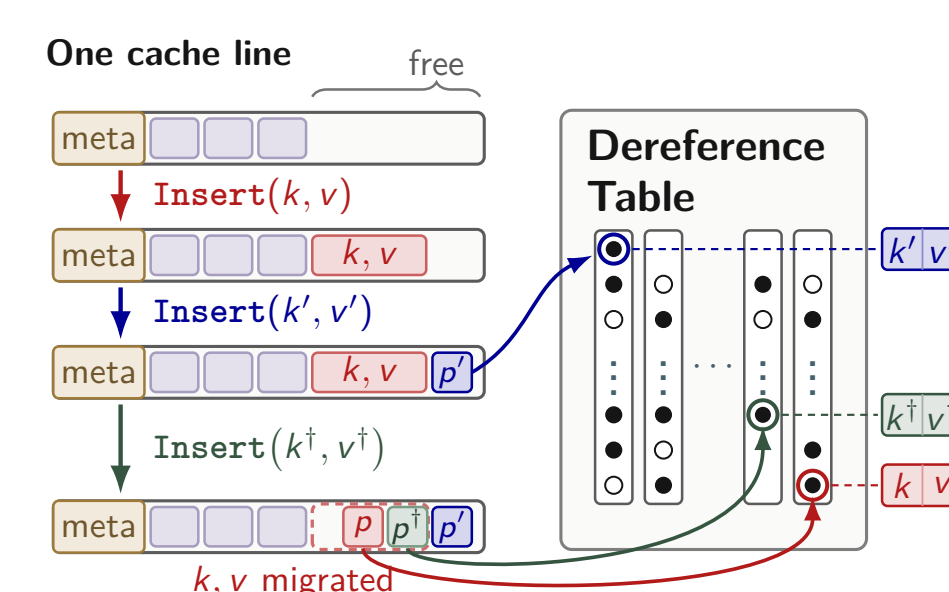
let neighboring quotient groups share one cache line.



Flattened-TPHT: one-hop overflow

A full home line migrates **one** tuple out, leaving a tiny pointer — overflow **never chains**.

- **1.25** memory accesses per operation.
- Migration is rare: **5.1%** of tuples.
- **31** pointer slots per line; max load **24** w.h.p.



Evaluation

Workload: YCSB, $2^{26}=64\text{M}$ 64-bit key–value pairs, 16 threads.

Baselines: libcuckoo^[1], IcebergHT^[2], Junction^[3], TBB^[4].

Machine: Intel Xeon Silver 4314, 16 cores, 24 MiB L3.

Key numbers

Result 1: throughput

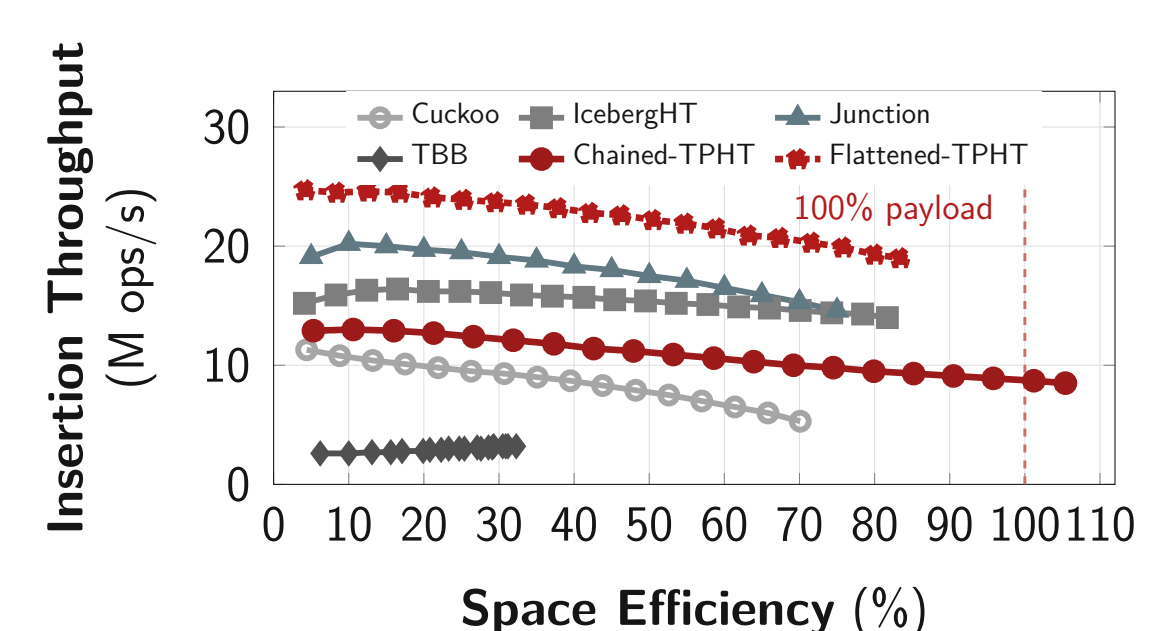
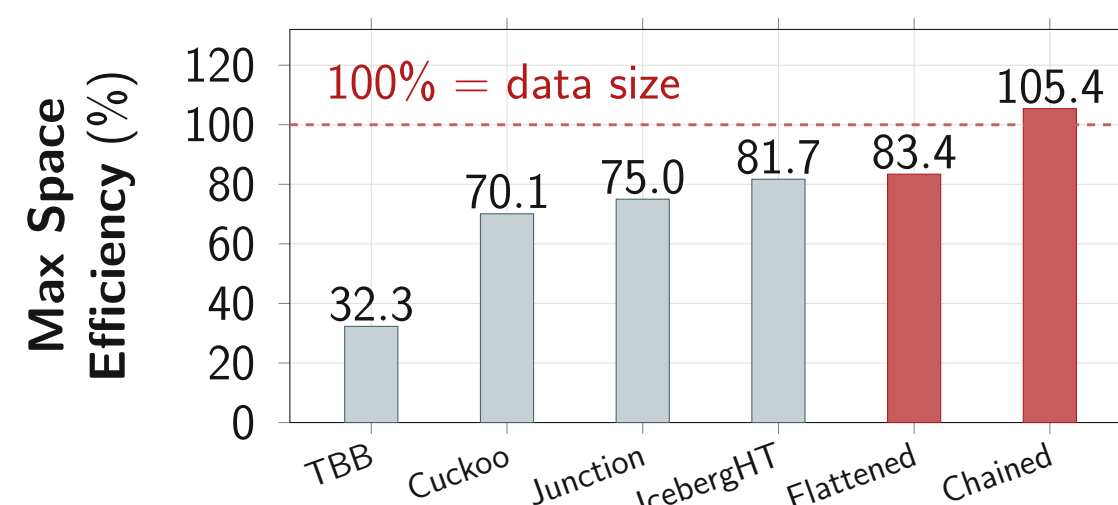
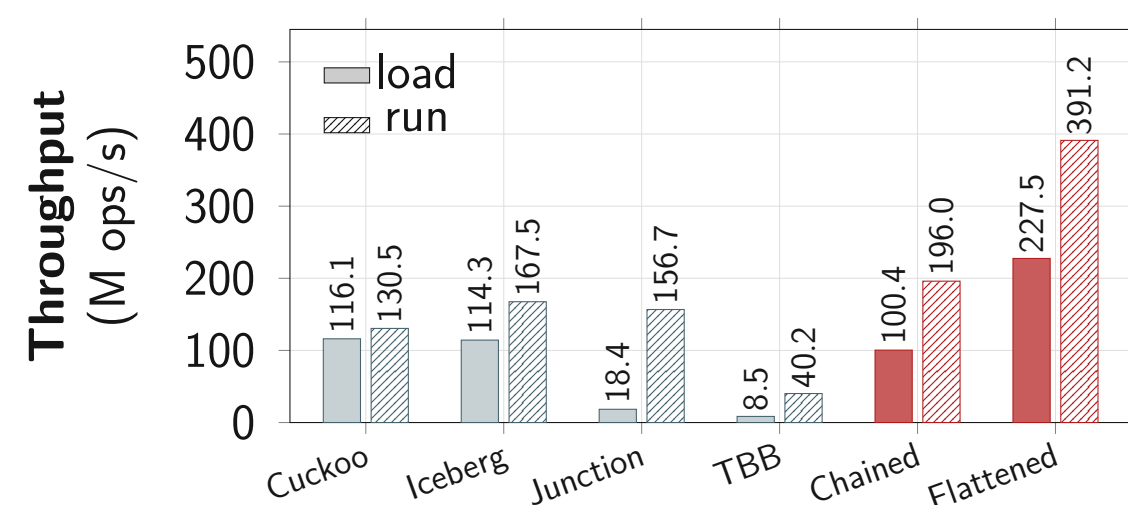
Result 2: space efficiency

Result 3: speed vs. space

105.4%
max space efficiency

2.3×
YCSB speedup vs. best

1.25
memory accesses / op



[1] Li, Andersen, Kaminsky, Freedman. *Algorithmic Improvements for Fast Concurrent Cuckoo Hashing*. EuroSys '14. [2] Pandey et al. *IcebergHT: High Performance Hash Tables Through Stability and Low Associativity*. SIGMOD '23. [3] Preshing. *Junction concurrent maps*. [4] Intel oneTBB concurrent_hash_map.